navAggregate

1 Introduction

The `navAggregate` flight software module combines multiple input navigation messages from different sources such as star trackers, EKF outputs, coarse sun sensors, and then combine them into a single attitude and translation navigation messages.

Each navigation source can have a full or partial set of navigation input fields. For attitude navigation, the input fields are time tag, attitude state, angular rate, and sun direction vector, while the translation navigation includes inputs of time tag, position, velocity, and accumulated ΔV.

Since different modules can produce similar navigation quantities, the user can choose which module is trusted the most for each field. This allows for a uniform solution that combines most reliable inputs of each field.

The number of input messages is defined through either `attMsgCount` or `transMsgCount`. If either of these values is zero, then the corresponding output navigation message is populated with zero components.

For each attitude and translational navigation field, the user must specify an index `Idx` that corresponds to the navigation message that will fulfill that field. The index is zero-based, so 0 means taking the field of the first navigation message. The Idx index must be less than input navigation message counter `attMsgCount` or `transMsgCount` respectively.

Note that the index name specifies which field is being selected, for e.g. `attIdx`, while the index value corresponds to the module that provides that field.

2 Module Input/Output Messages

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
`navAttInMsg`	`NavAttMsgF32Payload`	Input message containing the attitude navigation message.
`navTransInMsg`	`NavTransMsgF32Payload`	Input message containing the translational navigation message.
`navAttOutMsg`	`NavAttMsgF32Payload`	Output message containing the blended attitude navigation message.
`navTransOutMsg`	`NavTransMsgF32Payload`	Output message containing the blended translation navigation message.

3 Module Parameters

The following table lists all the module parameters than can be set. The parameters are optional unless indicated (if not specified default is used).

Table 2: Module Parameters

Parameter Name	Type	Units	Default	Description	Bounds
attTimeIdx	uint32_t	[-]	0	index of message to use for attitude message time	attTimeIdx < MAX_AGG_NAV_MSG (checked in setter)
transTimeIdx	uint32_t	[-]	0	index of message to use for translation message time	transTimeIdx < MAX_AGG_NAV_MSG (checked in setter)
attIdx	uint32_t	[-]	0	index of message to use for inertial MRP `sigma_BN`	attIdx < MAX_AGG_NAV_MSG (checked in setter)
rateIdx	uint32_t	[-]	0	index of message to use for attitude rate `omega_BN_B`	rateIdx < MAX_AGG_NAV_MSG (checked in setter)
posIdx	uint32_t	[-]	0	index of message to use for inertial position `r_BN_N`	posIdx < MAX_AGG_NAV_MSG (checked in setter)
velIdx	uint32_t	[-]	0	index of message to use for inertial velocity `v_BN_N`	velIdx < MAX_AGG_NAV_MSG (checked in setter)
dvIdx	uint32_t	[-]	0	index of message to use for accumulated ΔV	dvIdx < MAX_AGG_NAV_MSG (checked in setter)
sunIdx	uint32_t	[-]	0	index of message to use for sun direction in body frame	sunIdx < MAX_AGG_NAV_MSG (checked in setter)
attMsgCount	uint32_t	[-]	0	total number of attitude messages available as inputs	attMsgCount ≤ MAX_AGG_NAV_MSG (checked in setter)
transMsgCount	uint32_t	[-]	0	total number of translational messages available as inputs	transMsgCount ≤ MAX_AGG_NAV_MSG (checked in setter)

Where `MAX_AGG_NAV_MSG` is equal to 10.

4 Algorithm Flow

At every update cycle, the `navAggregate` module performs the following steps:

- It reads and stores the latest attitude and translational input messages, up to `attMsgCount` and `transMsgCount`, respectively.
- A temporary array is constructed for both attitude and translational message payloads to represent the available sources for each navigation field.
- The user defines the indices of each navigation field and the algorithm selects them to be written for the output message.
- The algorithm writes the final attitude and translational messages to be sent to the downstream FSW modules.

5 Controller Functions

Below is a list of functions that this flight software module performs: - Store multiple attitude `attMsgs[]` and translational `transMsgs[]` navigation message inputs, such that the module can access them at every update cycle. - Read the latest data of both navigation messages and store them in a local array `msgStorage`. - Assign for each navigation message field (time tag, MRPs, angular rate, sun vector, position, velocity, ΔV) which index will fulfill it based on what the user defined. - A single aggregated output message will be constructed for each of attitude and translational navigation messages. - The two blended navigation messages will then be written and sent to the output to be handled by the downstream FSW modules. - In case the `attMsgCount` or `transMsgCount` was set to zero by the user, the output navigation message will stay zero.

6 Module Assumptions and Limitations

- The number of navigation message inputs should not exceed the value assigned for `MAX_AGG_NAV_MSG`, which is 10 by default from the header file.
- The user must correctly define the value of `attMsgCount` and `transMsgCount` to ensure a valid number of input messages.
- The module does not account for the correctness of the input's reference frame. The user must ensure that all inputs are frame consistent before entering the `navAggregate` module.
- The index value should always be less than the message count. This is because the index have zero-based values, while the message count have one-based values.
- Multiple `msgStorage` instances can exist in the module. Each instance represent a single navigation source that contains 4 navigation fields. However, the fields are valid based on the indices selected by the user, otherwise they will remain zeros.

7 Test Description and Success Criteria

The navigation aggregate unit test is located in `algorithms\navAggregate\_tests\test_navAggregate.py`. This test verifies that the module works as expected for different sets of navigation messages by comparing the cpp module output with the expected truth values.

In this unit test, 4 navigation messages are created and filled with data, (`navAtt1Msg` and `navAtt2Msg`) for attitude and (`navTrans1Msg` and `navTrans2Msg`) for translational.

The test includes 16 different scenarios, where different pairs of (`numAttNav`, `numTransNav`) are tested with values ranging from 0 to 3.

These values correspond to the internal cpp message count and four cases are being tested.

Before covering the different cases, it is important to understand how the unit test works. The cpp module will output the aggregate message using the indices of each navigation message such as `(TransTimeIdx, AttIdx, RateIdx, DvIdx)`. The index value is assigned based on (numAttNav-1) or (numTransNav-1) when the condition (numAttNav $>$ 1) or (numTransNav $>$ 1) is true. On the other hand, the truth values output are set manually based on `numAttNav` and `numTransNav`.

- Case 1 (`numAttNav` or `numTransNav` == 0): This means no input message is provided, so the aggregate output message will be all zeros.
- Case 2 (`numAttNav` or `numTransNav` == 1): Here, 1 input message is available and the index will remain 0 as it does not satisfy the condition (numAttNav $>$ 1) or (numTransNav $>$ 1). The expected aggregate output message is `navAtt1Msg` or `navTrans1Msg` for both cpp and truth values.
- Case 3 (`numAttNav` or `numTransNav` == 2): For this case, 2 input messages are available. The value satisfies the upper condition, so the index will be 1 based on (numAttNav-1) or (numTransNav-1). The expected aggregate output message is `navAtt2Msg` or `navTrans2Msg`.
- Case 4 (`numAttNav` or `numTransNav` == 3): This is a situation where 3 input messages are assumed available, but only 2 real messages actually exist. The test will set the cpp message count to 2 and

based on the condition (numAttNav-1), the index will be 2. Since this index has no valid data, the output of the message fields are all zeros. This case ensures that the module will not create fictional data for non existing input messages.

For the success criteria, the test records the aggregate outputs of each translational and attitude messages and then checks if they match the cpp output and the expected truth values with a tolerance of 10^{-7}. The test is considered successful if the output fields match exactly the selected input messages for valid indices and output zero values when no valid inputs or indices are assigned.